Del objetivo de negocio a un sistema de ML operable

ASUM-DM · Spec-Driven Development · skills ECC · OpenCode + Ollama · MLOps

Caso: churn en telecomunicaciones · 3 horas · 100 % local, sin tokens ni cuentas



Los proyectos de ML fallan por desconexión, no por el algoritmo



Negocio

Se predice algo que nadie puede accionar o sin KPI de éxito



Datos

Calidad no validada; datos de entrenamiento ≠ datos reales



Despliegue

El modelo vive en un notebook; no hay API ni versión



Monitoreo

El mundo cambia (drift) y nadie lo ve: degradación silenciosa

El remedio de hoy: un hilo de trazabilidad desde el KPI de negocio hasta la métrica que vemos en producción.

El mapa del día: cada pieza conecta con la siguiente



ASUM-DM

¿Qué problema de negocio y cómo sabremos que funcionó?

`business/ASUM-DM.md`

SDD · Spec Kit

¿Qué debe hacer el sistema y por qué? Luego, cómo.

`constitution · spec · plan · tasks`

Skills ECC

Buenas prácticas reutilizables que el agente carga

`.opencode/skills/`

OpenCode + Ollama

Agente open source con modelo local en VS Code

`opencode.json`

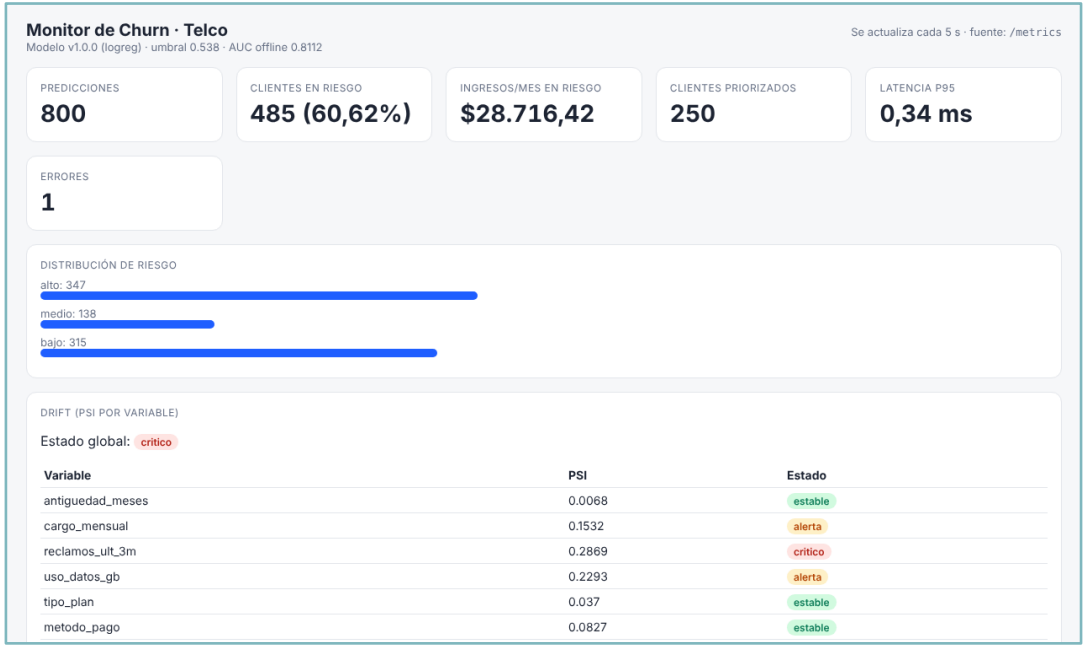
MLOps

Versión, health, métricas, drift y gobernanza

`/health · /metrics · drift`

KPI → criterio de éxito (SC) → requisito (FR) → tarea (T) → código → prueba → métrica en producción

Resultado final: lo que tendrán funcionando hoy



Captura real del dashboard del repositorio solución tras simular el escenario «crisis»

API de churn

POST /predict → probabilidad, nivel de riesgo y acción

Dashboard simple

/metrics: operación, negocio y drift (PSI)

Trazabilidad

ASUM-DM → spec → tareas → pruebas → model card

ASUM-DM: ciencia de datos con despliegue incluido

1. Entendimiento del negocio
2. Entendimiento de los datos
3. Preparación de datos
4. Modelado
5. Evaluación
6. Despliegue

Analytics Solutions Unified Method (IBM) amplía CRISP-DM con despliegue, operación y gestión de proyecto. Hoy: las fases 1 y 6 marcan el inicio y el final del hilo.

MLOps: el modelo es un producto que se opera



Despliegue

API versionada



Monitoreo

health, latencia, errores



Drift

datos y concepto (PSI)



Reentrenamiento

disparadores claros



Gobernanza

quién aprueba, evidencias



Trazabilidad

versión en cada predicción

Sin MLflow ni Kubernetes: model card JSON + logs JSONL + FastAPI enseñan los mismos conceptos en una laptop.

SDD: primero la especificación, después el código

1	2	3	4	5	6	7
`/speckit.constitution`	`/speckit.specify`	`/speckit.clarify`	`/speckit.plan`	`/speckit.tasks`	`/speckit.implement`	`/speckit.converge`
Principios	**Qué y por qué**	**Ambigüedades**	**Cómo (tecnología)**	**Tareas**	**Código**	**Brechas**

Regla de oro: la spec describe QUÉ necesita el usuario y POR QUÉ. La tecnología aparece por primera vez en el plan.

implement ⇄ converge se repite hasta que converge reporta «Converged»: el código queda alineado con la spec, no con la memoria del desarrollador.

Skills estilo ECC: experiencia reutilizable para el agente

`ml-churn-pipeline`

Reproducibilidad, sin fuga de datos, métricas de negocio

`python-testing`

TDD con pytest y TestClient

`api-security-review`

Validación, errores, privacidad

`mlops-architecture`

Versión, health, métricas, drift

`docs-es`

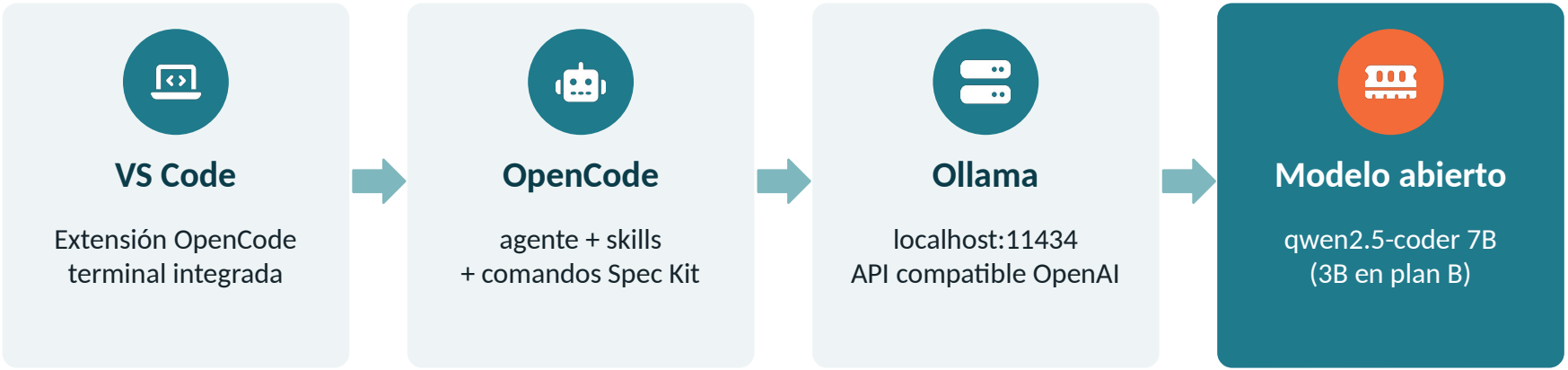
README ejecutable y gobernanza

`spec-code-review`

Matriz requisito → código → prueba

Archivo: .opencode/skills/<nombre>/SKILL.md con name y description. Inspiradas en Everything Claude Code (ECC).

OpenCode + Ollama: un agente que no sale de tu laptop



- Privacidad: código y datos no se envían a terceros
- Funciona sin internet una vez instalado
- Cero tokens, cero cuentas, cero costo por uso

Advertencia ética: la retomamos en cada bloque

Sesgo

La edad no entra al modelo, pero el plan prepago puede ser su proxy. Auditamos el impacto por grupo (≥ 0,80).

Privacidad

Rechazamos campos no previstos (p. ej., cédula). Los logs guardan un hash, nunca el ID.

Automatización

El modelo prioriza a quién OFRECER beneficios. Nunca para negar servicio o subir precios. Revisión humana siempre.

Datos 100 % sintéticos. Con datos reales aplique la normativa de protección de datos (en Colombia, Ley 1581 de 2012).

Verificación de instalación: luz verde o pareja

1. **Abrir VS Code en taller-churn-mlops y activar .venv**

```
source .venv/bin/activate   |   .venv\Scripts\Activate.ps1
```

2. **Smoke test**

```
python scripts/smoke_test.py
```

3. **Abrir OpenCode y elegir el modelo local**

```
opencode  →  /models  →  taller-coder
```

4. **Probar el agente**

```
Hola, responde en español en una línea.
```

¿Falla algo? Se empareja YA con alguien en verde y sigue. Plan B: modelo 3B, servidor Ollama compartido o modo pareja.

Bloque 1 · Del objetivo de negocio al problema de ML

Elemento	Definición en el caso churn
Objetivo de negocio	Reducir la cancelación mensual actuando antes de que el cliente se vaya
KPIs	Tasa de cancelación · ingresos retenidos · costo de campaña (USD 8 × contacto)
Problema de ML	Clasificación binaria: probabilidad de cancelar en 30 días
Unidad de análisis	Cliente-mes
Datos (simulados)	antigüedad, plan, facturación, reclamos, uso de datos, pago, renovación
Despliegue	API tiempo real + lote mensual · reentrenamiento mensual o por drift

Entregable: business/ASUM-DM.md completo y revisado por la pareja vecina (25 min)

Criterios de éxito: el umbral se decide con costos

≥ 0,70	≥ 0,40	≥ 0,75	≥ 0,80
Recall	**Precision**	**ROC-AUC**	**Impacto dispar**
detectar a quien se va	no quemar presupuesto	capacidad de ordenar	equidad por edad

Falso negativo
Cliente que se va sin contacto: se pierden ~12 meses de facturación.

Falso positivo
Contacto innecesario: USD 8 y un cupo menos de los 250 del mes.

Revisión por pares: ¿objetivo medible? ¿recall vs precision justificado con costos? ¿riesgo de uso indebido con mitigación?

Bloque 2 · Spec Kit: qué genera cada comando

`/speckit.constitution`	`.specify/memory/constitution.md`	calidad, seguridad, explicabilidad, pruebas, gobernanza
`/speckit.specify`	`specs/001-*/spec.md`	historias, FR, SC medibles, casos límite
`/speckit.clarify`	`spec.md (sección Clarificaciones)`	preguntas y decisiones
`/speckit.plan`	`plan.md, research.md, contracts/`	FastAPI, scikit-learn, pandas, pytest
`/speckit.tasks`	`tasks.md`	tareas por historia, [P], archivo, FR
`/speckit.checklist`	`checklists/*.md`	validación de requisitos

Prompts completos en prompts/PROMPTS_OPENCODE.md (P2.1 a P2.7). Setup: specify integration install opencode

¿Spec o solución prematura? Primero el qué y el porqué

No: solución prematura

«Crear un endpoint FastAPI /predict que cargue un RandomForest con joblib, valide con Pydantic y guarde las métricas en MLflow.»

Decide la tecnología antes de entender al usuario y esconde los criterios de éxito.

Sí: especificación

«Como analista de retención quiero consultar el riesgo de un cliente para decidir si le ofrezco un beneficio.»
FR-002: rechazar datos inválidos indicando campo y motivo.
SC-001: detectar ≥ 70 % de quienes cancelan.

Ejercicio (P2.4): pidan al agente listar cada tecnología mencionada en su spec.md y reescribirla como necesidad de negocio.

Del requisito a la prueba: lo que verifica /speckit.converge

Req.	Qué	Tarea	Código	Prueba
FR-002	Validar rangos y dominios	T011	`api.Cliente (Field, Literal)`	`test_rechaza_valores_fuera_de_rango`
FR-003	Rechazar datos no previstos	T011	`extra="forbid"`	`test_rechaza_..._campos_extra`
FR-008	Log auditable sin ID en claro	T019	`monitoring.seudonimizar`	`test_log_no_guarda_id_en_claro`
SC-001	Recall ≥ 0,70	T008	`train.entrenar + umbral`	`test_model_card_cumple_criterios`

Plan (P2.5): ahora sí entra la tecnología — Python 3.11, pandas, scikit-learn, FastAPI, pytest, JSON/JSONL, Docker opcional. Tareas (P2.6): cada una con archivo destino y FR/SC.

Bloque 3 · Muestro, ejecutan, revisamos, convergemos



Muestro

prompt corto (1 min)



Ejecutan

el mismo prompt (3–4 min)



Revisamos

código vs spec.md (1 min)

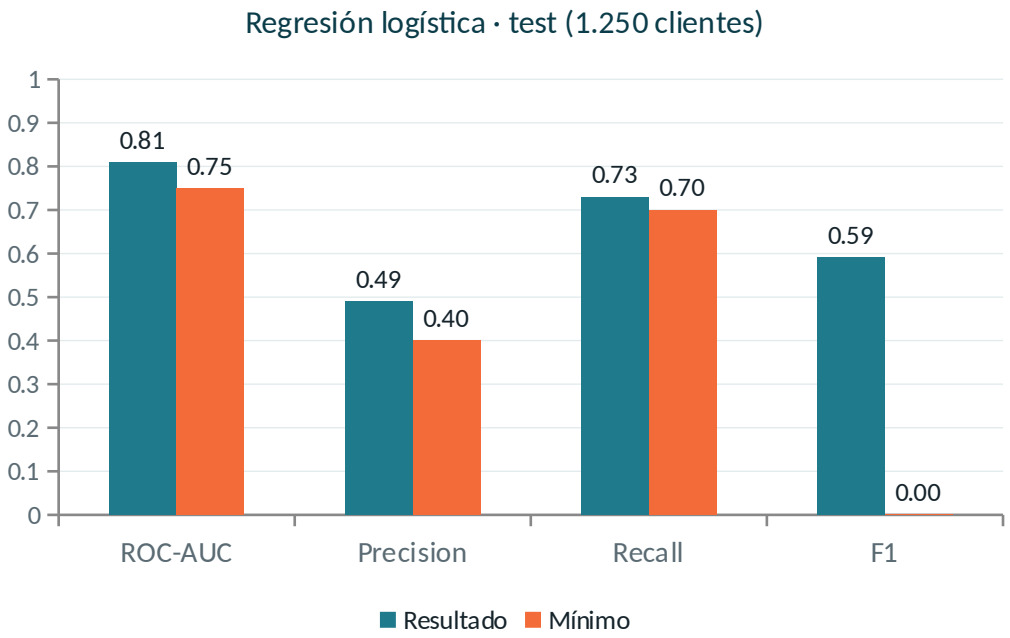


Convergemos

/speckit.converge si hay brechas

a. Dataset sintético	**b. Preparación de datos**	**c. Baseline (LogReg / RF)**	**d. Evaluación**
e. Serialización + model card	**f. API FastAPI /predict**	**g. Pruebas pytest**	**h. README ejecutable**

Baseline: cumple los criterios de ASUM-DM



0,54

umbral calibrado con CV en train (máx. precisión con recall ≥ 0,70)

234 / 319

clientes que cancelan detectados (TP / positivos)

0,90

ratio de impacto dispar por edad (mínimo 0,80)

API con validación estricta y errores que se entienden

```
POST /predict
{ "antiguedad_meses": 6,
  "tipo_plan": "prepago",
  "cargo_mensual": 45.5,
  "reclamos_ult_3m": 3, ... }

200 OK
{ "probabilidad_churn": 0.97,
  "nivel_riesgo": "alto",
  "recomendacion": "Contactar en 48 h…",
  "version_modelo": "1.0.0",
  "aviso": "requiere revisión humana" }
```

```
{ "cargo_mensual": -10 }

422
{ "error": "Entrada inválida",
  "detalle": [{ "campo": "cargo_mensual",
     "mensaje": "Input should be >= 0" }] }
```

`extra="forbid"`
rechaza campos como «cedula»
`503` si el modelo no cargó · **`500`** genérico sin stack trace

Bloque 4 · MLOps mínimo viable sin infraestructura



`/health`

status, modelo cargado, versión, aprobado_para_despliegue



`/metrics`

volumen, errores, latencia p95, distribución de riesgo



`logs/predicciones.jsonl`

una línea JSON por predicción, ID con hash



`model_card.json`

versión, fecha, hash del dataset, métricas, equidad



Métricas de negocio

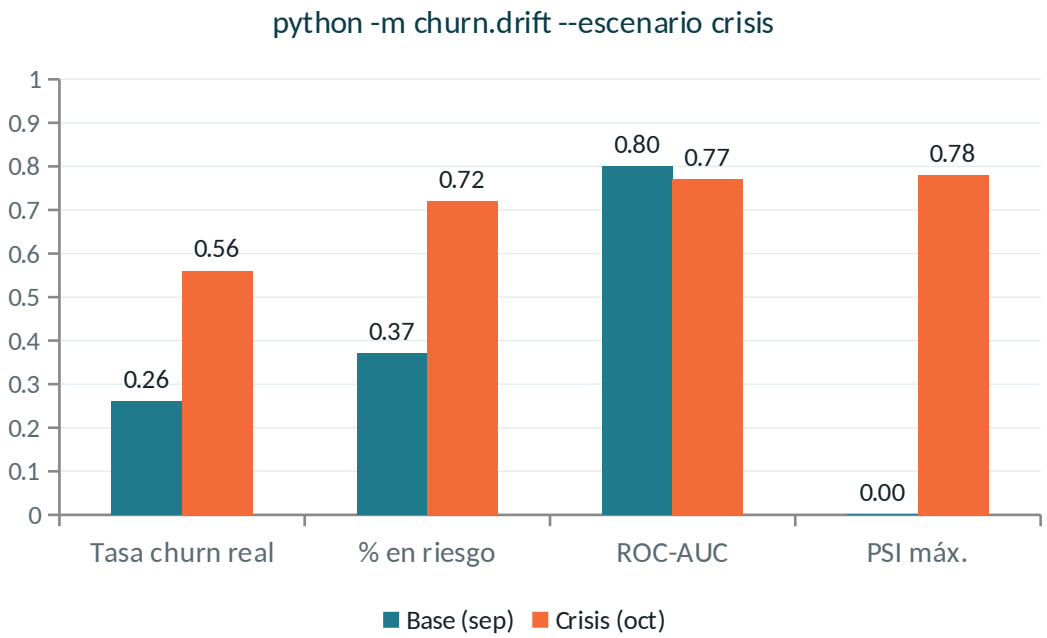
clientes en riesgo, ingresos en riesgo, priorizados, ROI



`docs/GOBERNANZA.md`

RACI, evidencias, versionado semántico

Drift: el modelo no se rompe, se degrada en silencio



×2,1

ingresos mensuales en riesgo (99 k → 208 k USD), pero el presupuesto sigue siendo 250 contactos

PSI ≥ 0,25

en reclamos y cargo: estado crítico → reentrenar y revalidar

¿Qué decisión toman? ¿Quién la aprueba?

Gobernanza: quién aprueba, qué se guarda, cómo versionar

Actividad	Analítica	Retención	TI	Riesgo
Objetivo, KPIs y criterios	R	A	C	C
Entrenar y evaluar	A/R	I	I	C
Auditoría de sesgo y privacidad	R	C	I	A
Aprobar paso a producción	R	C	C	A
Operar API y logs	C	I	A/R	I
Reentrenar o retirar	R	C	C	A

Evidencias
model card · evaluacion.json · drift.json · pytest · spec/tasks · acta

Versión
MAJOR: cambia objetivo o features · MINOR: reentrenamiento · PATCH: sin efecto en predicciones

Reentrenar si
PSI ≥ 0,25 · recall < 0,70 · cada mes

Demo final en 6 minutos: el hilo completo



Negocio	Spec	Código	API	Prueba	Métricas	Drift
ASUM-DM.md	spec → plan → tasks	src/churn/	/predict ok y 422	pytest verde	/dashboard	escenario crisis

Pregunta para cada demo

Si mañana el KPI de churn empeora, ¿qué archivo abren primero y qué evidencia muestran para decidir?

Rúbrica: 5 criterios, 4 niveles, aprobado ≥ 12/20

1	**Alineación negocio-ML**	KPIs con fórmula, criterios y umbral justificados con costos	1 · 2 · 3 · 4
2	**Calidad de especificación**	Sin solución técnica, FR verificables, SC medibles	1 · 2 · 3 · 4
3	**Calidad y pruebas del código**	Validación, errores, pytest verde, README ejecutable	1 · 2 · 3 · 4
4	**Trazabilidad MLOps**	health, metrics, logs, model card, drift y gobernanza	1 · 2 · 3 · 4
5	**Uso responsable de IA**	Sesgo auditado, privacidad, revisión humana y crítica al código generado	1 · 2 · 3 · 4

Uso responsable · reto para casa

El modelo apoya, las personas deciden

Sin edad como predictor y con auditoría por grupo · IDs con hash · nunca para negar servicio o subir precios · revisión humana en toda acción.
Reto: adapta el pipeline a fraude, inventario o deserción estudiantil (docs/RETO_CASA.md).


Equidad


Privacidad


Revisión humana


Reto